Skepticism Needs a Second Source

How Tool-Using Financial LLM Agents Handle Silent Data Corruption

Rayansh Singh
Independent Researcher
rayansh365@gmail.com

Abstract

Tool-using LLM agents in finance read prices, rates and risk figures from data services that can fail silently. The call succeeds and returns a well-formed value that is stale, mis-scaled or slightly wrong. We present FinAgentFault, a harness that injects seeded, logged silent faults into deterministic financial tools, and use it for a paired study of six open-weight models on 131 tasks. When a fault changes the correct answer, the plain agent reports the wrong value without warning 89% of the time, and a verification prompt barely changes this (86%). Most of these faults cannot be seen from a single source. An uncorrupted second source helps far more. On five models, pass-through falls from 90% to 46% for the plain agent and from 84% to 20% with the prompt. The remaining failures come from models that never query the second source, and from one that queries it and still reports the corrupted value. A scripted check that compares both sources leaves no unsafe answer, though only because our second source is never wrong. Skepticism needs something to check against. A manual audit also shows that keyword scoring overstated explicit-failure errors by a factor of 2.5 to 3.

Keywords

LLM agents, tool use, silent data corruption, robustness, financial AI, evaluation

1 Introduction

An LLM agent that answers "what was the fund's 95% VaR on January 5?" calls a risk service and reports what comes back. When that service fails loudly, with a timeout, an HTTP 429 or unparseable JSON, the agent can see the failure and retry or decline. Data systems also fail without saying so. A cached quote is served under today's date, a price arrives in cents instead of dollars, a value is shifted by a bad join. The response parses, passes schema validation and looks plausible. Hardware engineers call the analogous problem silent data corruption [7]. In this case nothing in the agent's context signals that anything went wrong, and in finance the wrong number is the deliverable.

Recent benchmarks have begun to inject faulty tool outputs into agent loops [11, 28, 40]. ToolMaze in particular reports that semantically corrupted outputs degrade agents far more than explicit errors [40]. These studies establish that the problem exists. They leave three questions open that matter for deploying an agent on financial data. How often does a corrupted value pass straight through to the answer, as opposed to being harmless or caught? Does instructing the agent to be skeptical help, and by how much at most? And what does it take for verification to work, given that verification needs something to check against?

We answer these with a deliberately simple, tightly controlled design. FinAgentFault wraps deterministic financial tools with an injector whose faults are seeded, logged and shared across models, so every comparison is paired at the level of individual runs. We run two agents, a plain ReAct loop (A) and the same loop with a verification prompt (B), on six open-weight models under four conditions: clean tools, explicit failures, silent faults, and silent faults with an uncorrupted second source. A scripted policy with no model in it serves as a reference for what harness-level verification achieves. Because the fault log records exactly what each agent received, we can also replay every run and determine whether the fault actually changed the correct answer.

Our findings are as follows.

- **Silent faults that matter pass through.** In about half of the faulted runs, the fault changes the correct answer. In those runs the plain agent reports the corrupted value without warning 89% of the time, and 79 to 100% for each of the six models.
- **A verification prompt alone helps little, and could not help much.** It lowers the pass-through rate from 89% to 86% (paired difference −3.1 points, 95% CI −6.5 to 0.0). Four in five answer-changing faults are stale values or 2% perturbations that a single source cannot reveal. On the remaining fifth, unit-scale errors that a plausibility check could catch, the agents still pass the corrupted value through 48% (A) and 45% (B) of the time, with no significant prompt effect.
- **A second source helps far more.** On the five models with this condition, pass-through falls from 90% to 46% for the plain agent and from 84% to 20% for the prompted agent. The prompted agent is now significantly safer than the plain agent (paired difference −12.5 points), and stays so when any one model is dropped.
- **Failures arise at two stages, and a harness can remove both.** Some models fail to acquire the second source and one fails to reconcile it with the first. A scripted policy that queries both sources and compares them leaves no unsafe answer, and a trace-level monitor catches all 26 reconciliation failures. Both results hold because our second source is never corrupted, so they measure what enforcement costs, not how it fares against bad evidence.
- **Keyword scoring of abstentions is fragile.** A manual audit, complete for two conditions and sampled for the third, reverses two per-model conclusions we had drawn during development but leaves the main results intact.

Our reading is that, in these agents, instruction without independent evidence achieves little, while instruction combined with evidence helps. This mirrors work showing that LLMs rarely self-correct without external feedback [12] but improve when a tool supplies it [9]. We will release the harness, tasks, fault policies, traces and audit labels upon acceptance.

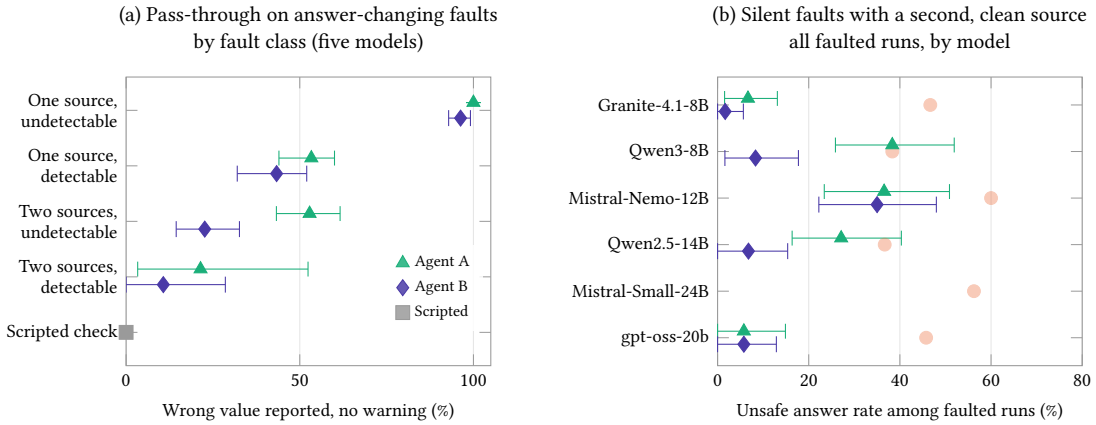


Figure 1: Most single-source faults cannot be seen, and a second source helps. (a) Share of answer-changing faults that end in a wrong, unflagged answer, split by whether a plausibility check could catch the fault in principle (×100 unit errors) or not (stale values and ±2% perturbations), for runs paired by model, task and seed. The scripted check queries both sources itself and has no interval, since it makes no model calls and its zero follows from the clean second source. (b) Unsafe answer rate among all faulted runs for each model with a second source. The faint orange dot is the same model's plain agent without it. The prompt (Agent B) adds a large further drop for Qwen3-8B and Qwen2.5-14B, the two models whose plain agent rarely calls the second source. Mistral-Small-24B has no data for this condition (Section 4). Bars are 95% task-clustered bootstrap intervals under automatic scoring.

Table 1: Headline quantities. Pass-through is the share of answer-changing faulted runs whose answer reports the wrong value with no warning. Brackets are 95% task-clustered bootstrap intervals. The last column says how each quantity was scored.

Quantity	Value (%)	Scoring
Share of answer-changing silent faults that a plausibility check could catch	20.5 (33/161)	Replay
Pass-through, plain agent A, silent, 6 models	89.4 [80.3, 97.9]	Automatic
Pass-through, prompted agent B, silent, 6 models	85.8 [76.1, 94.3]	Automatic
Pass-through on the detectable faults only, A / B	48.5 [42.3, 53.8] / 45.5	Automatic
Pass-through, plain agent A, second source, 5 models	46.3 [35.2, 57.6]	Automatic
same, after manual audit	43.3 [33.6, 52.9]	Audited
Pass-through, prompted agent B, second source, 5 models	19.6 [12.5, 28.2]	Automatic
same, after manual audit	15.9 [9.8, 22.9]	Audited
Scripted harness check (no model), second source	0.0 (0/28)	Deterministic

2 Related Work

Tool-using agents. ReAct [35] and Toolformer [27] established the pattern of interleaving reasoning with tool calls. ToolLLM [23], AgentBench [17] and τ-bench [34] measure whether agents choose and use tools correctly, assuming the tools return correct data. FinAgentFault keeps tool use simple and varies the correctness of the data instead.

Unreliable tool outputs. Indirect prompt injection places attacker instructions in content an agent retrieves [5, 10, 38]. ToolEmu [26] and ToolSword [36] study risky actions and harmful tool feedback, RoTBench [37] perturbs tool documentation, and ToolBeHonest [39] tests whether agents recognize unsolvable tasks. Closest to our work, ToolMaze [40] crosses explicit and implicit (semantically corrupted) failures with transient and permanent ones across six domains including finance, and finds a large gap between the two kinds of failure and only partial gains from a failure-aware prompt. AgentNoiseBench [28] injects erroneous and partial tool outputs into existing benchmarks, and AgentProp-Bench [11] measures how often a corrupted parameter propagates to a wrong final answer. Our silent-fault result agrees with these findings. We add run-level pairing across models and agents, a replay that separates harmful faults from harmless ones, and a crossed comparison of a verification prompt with and without a redundant source.

Knowledge conflicts. LLMs tend to adopt coherent external evidence that contradicts their parametric knowledge [18, 33]. RGB reports that models rarely detect counterfactual facts in retrieved passages even when warned [3], and ClashEval finds that models adopt a perturbed retrieved value less often the further it departs from the truth [30]. Our fault-type results show the same gradient inside a tool loop: a ×100 unit error is sometimes noticed, and a 2% perturbation never is when only one source exists.

Verification and abstention. Intrinsic self-correction often fails without external feedback [12], whereas tool-grounded critique [9] and planned verification questions [6] can reduce errors. Failure-Transparent Agents [42] find that a transparency instruction alone more than halves false claims of success after a tool failure, and that requiring structured evidence nearly eliminates them. There

the failure is visible to the agent and the instruction only has to make it report it. Our Agent B tests the cheapest version of this idea, an instruction in the system prompt, against failures the agent cannot see. Scoring abstention is itself hard [29]. AgentProp-Bench reports near-chance agreement between substring-based scoring and human labels [11]. Our audit finds the same problem under explicit failures, where abstentions are worded freely, but far less under silent faults.

Financial LLM evaluation. FinQA [4], TAT-QA [41], FinanceBench [14] and the Finance Agent Benchmark [1] evaluate reasoning and research over financial documents, and BloombergGPT [31] targets financial text. These treat the source data as correct. We hold the reasoning demand low on purpose so that differences in outcome can be attributed to the data fault.

3 FinAgentFault

3.1 Tasks and tools

The benchmark has 131 tasks in three families, each served by one deterministic tool that reads a local JSON snapshot: market data (46 tasks, closing prices of AAPL, MSFT and GOOGL), macroeconomic data (45 tasks, fed funds rate, unemployment rate and CPI) and portfolio risk (40 tasks, 95% VaR, Sharpe ratio and holding weights for three portfolios). Dates fall in January 2024. Ninety-one tasks ask for a single value and 40 ask whether a quantity increased between two dates, which requires two tool calls. Ground truth is produced by calling the clean tools, never typed by hand. The snapshots are synthetic: values are realistic in scale but are not guaranteed to match historical records. This keeps evaluation reproducible and offline at the cost of realism (Section 7).

3.2 Fault model

A **silent fault** returns a schema-valid response with a wrong value. The injector wraps each tool and, on every successful call, rolls each entry of a YAML fault policy against a seeded random generator. Four types are used, with these per-call probabilities. **Stale data** (0.10) replaces values with those of the previous valid period while keeping the requested date in the response. **Unit scale** (0.05) multiplies a price by 100, as if cents were reported as dollars. **Small perturbation** (0.10) moves a price by ±2%, a value that is plausible in isolation (this type is named `contradiction` in the released configs). **Partial result** (0.10) drops 30% of the rows of a list-valued field. Unit-scale and perturbation faults target the price field and therefore only fire on market-data calls. Several faults can fire on one call.

An **explicit failure** makes the call raise instead: an internal exception, a timeout, an HTTP 429, or a malformed-response error, each with probability 0.075 per call. The agent receives the error text as the tool result and may retry.

Every injected fault is logged with the original and corrupted response, fault type, parameters, severity, timestamp and seed. Each run's injector seed is derived from the sweep seed and the task identifier, so for a given seed and call sequence the same fault fires for every model and both agents. This makes the A-versus-B comparison paired at the run level. Each injector draws one random number per applicable policy entry for every primary-tool call, indexed by the order of those calls within a run. Calls to the secondary tool consume no draws, because no policy entry targets it. We checked the pairing on the stored runs. Among A-versus-B pairs in which at least one run was faulted, 323 of 334 (96.7%) in the silent condition and 278 of 282 (98.6%) in the redundant-source condition received identical fault sets. The rest are pairs whose primary-call sequences differ.

Faults that change the answer. A fault can fire without affecting the correct answer: a stale value may equal the current one, a 2% perturbation may not flip the direction of a comparison, and a dropped holdings row may not be the one asked about. For each faulted run we apply the logged corruptions to the task's clean tool calls and compute the answer an agent would give by faithfully reporting what it received. We call the fault **answer-changing** if that answer is wrong under the scoring tolerance (Section 3.4). The replay assumes that every fault hit a request the clean trace also makes. This holds in 1,237 of the 1,240 faulted silent and redundant-source runs.

3.3 Conditions and agents

We use four conditions from the FinAgentFault design: **Clean** (no faults), **Explicit** (explicit failures only), **Silent** (silent faults only) and **Redundant source** (silent faults on the primary tool, plus a second tool named `<tool>_secondary` that serves the same uncorrupted data and is described to the agent as "an independent data source for the same information, use it to cross-check"). Faults never target the secondary tool, so whenever the primary is corrupted, a disagreement exists for the agent to find. This is a best case for redundancy. Note that the tool description itself invites cross-checking, so Agent A in this condition is not free of instruction. The condition therefore changes two things at once, the availability of a second source and a description that points to it. It is also asymmetric in a way that favors any policy that prefers the second tool. Because the secondary is never wrong, an agent that always trusts it is indistinguishable from one that verifies (Section 5.4), and the gains we report are an upper bound on what an imperfect second source would give.

Agent A is a ReAct-style tool loop with no system prompt of its own (some chat templates insert a generic default). Agent B is the same loop with a system prompt (Appendix A) stating that tool output can be well-formed and still wrong. It asks the agent to sanity-check high-impact numbers, call a second tool when one can supply the same fact, and say plainly what is uncertain. The prompt describes a policy and does not script wording. Both agents decode greedily (temperature 0) with at most ten steps. The sweep seed therefore varies the fault draws, not the decoding. Each model call allows up to 1,024 new tokens and we set no other sampling parameter. The client sends no reasoning setting, so gpt-oss-20b runs at the server's default reasoning effort.

3.4 Metrics and scoring

A run is **faulted** if at least one fault fired during it. The **unsafe answer rate** is the fraction of faulted runs whose final answer is wrong and carries no warning. The **fault detection rate** is the fraction of faulted runs whose answer flags a problem. We also

Table 2: Models and runs. Faulted counts are over both agents and both seeds. Excluded is the share of attempted runs dropped because the serving stack returned an error rather than an answer.

			Faulted runs			
Model	Params	Format	Silent	Expl.	Red.	Excl. %
Granite-4.1-8B	8B	FP16	120	188	120	0.0
Qwen3-8B	8B	FP16	120	188	120	0.0
Mistral-Nemo-12B	12B	AWQ INT4	120	188	112	1.1
Qwen2.5-14B	14B	AWQ INT4	120	193	118	0.0
Mistral-Small-24B	24B	W8A8 INT8	64	153	–	26.0
gpt-oss-20b	21B MoE	MXFP4	122	194	104	2.9

report task accuracy and the **false alarm rate**, the fraction of unfaulted runs whose answer flags a problem.

Correctness is scored automatically: numeric answers by tolerance (relative 10^{-3}, absolute 10^{-2}), boolean answers by yes/no and direction language. Under this tolerance a value within 0.1% of the truth counts as correct, so a stale closing price that is off by $0.27 is not an error. A flag is detected by matching the answer against a fixed list of hedging and failure-report phrases after case and typographic normalization. This detector is a transparent keyword heuristic, not a semantic classifier. We developed the phrase list while inspecting these runs, froze it before computing the results reported here, and measure its remaining errors with a manual audit (Section 5.7).

3.5 Statistics

All models are analysed on the same two sweep seeds. Trials that share a task are correlated across seeds and models, and with derived seeds they also share fault draws. Pooled 95% intervals therefore come from a task-clustered percentile bootstrap [8] that resamples the 131 task identifiers with replacement (2,000 resamples). Agent A versus Agent B comparisons use runs paired by model, task and seed, restricted to pairs in which a fault fired for both agents. For these we report the paired difference in unsafe rate with a task-clustered interval, and an exact two-sided McNemar test [19] on the discordant pairs.

4 Experimental Setup

We evaluate six open-weight instruction-tuned models (Table 2): Granite-4.1-8B [13], Qwen3-8B with thinking disabled [25], Mistral-Nemo-12B [20] and Qwen2.5-14B [24] with AWQ 4-bit weights [16], Mistral-Small-24B with W8A8 integer quantization [21, 32], and gpt-oss-20b, a mixture-of-experts model released in MXFP4 [22]. They span four model families and four numeric formats. Each model is served with vLLM [15] and its native tool-call parser, on two T4 GPUs for the first four and one A100 for the last two. Each model runs the clean condition once and every other condition with two seeds, for both agents, giving 10,067 scored runs. Appendix D lists the exact checkpoints.

Excluded runs. When the serving stack returns an error instead of a response, we log the run and exclude it rather than score it. Exclusions are rare for five models. For Mistral-Small-24B they are not: vLLM's Mistral tool-call parser fails deterministically on the second sequential tool call in a conversation, independent of the chat template, so most two-call comparison tasks never complete. We report its clean, silent and explicit results and drop its redundant-source condition, where 78% of Agent B runs were excluded. Its silent results rest entirely on single-value tasks. Its explicit results include 55 comparison runs, but 53 of them stopped after one call, because runs that attempted a second call were excluded. Exclusion therefore depends on the agent's own behavior, so its explicit comparison results are a selected subset. All redundant-source comparisons below therefore use the other five models, and we recompute the matching silent-condition baselines on the same five.

5 Results

5.1 Silent faults pass through to the answer

Under silent faults, the plain agent gives a wrong and unflagged answer in 46.5% of faulted runs (95% CI 34.5 to 58.9, pooled over six models). Under explicit failures it does so in 7.3% (5.1 to 9.8), a difference of 39.2 points (26.9 to 51.9). The direction holds for every model, and for every model the two per-model intervals do not overlap (Figure 1a, Table 3). This gap is partly expected by construction. Under an explicit failure the agent never receives a wrong value, so an unsafe answer requires it to invent one. The comparison is still the operationally relevant one, since it holds the tool, the tasks and the agent fixed and changes only whether the failure is visible.

The more informative question is what happens when the agent does receive a wrong value. Of the 331 faulted plain-agent runs, 161 (49%) carried an answer-changing fault. In those runs the agent reported the corrupted answer without warning 89.4% of the time (80.3 to 97.9), flagged a problem 1.2% of the time (0.0 to 4.2), and recovered the correct answer in the rest. The per-model rate ranges from 79% (Qwen2.5-14B) to 100% (Granite-4.1-8B). In the other 170 faulted runs, where the fault did not change the answer, only 5.9% ended unsafe. The 46.5% headline figure is therefore a mixture of two very different situations. A fault that matters is almost always passed through, and a fault that does not matter is almost always harmless. Answer-changing faults were more common in single-value tasks (126 of 192) than in comparisons (35 of 139), where a stale or perturbed value usually leaves the direction of change intact.

The failures are not explained by weak task ability. On clean tools, accuracy is 90 to 100% for every model and agent, and no plain agent raises a false alarm. Nor does the agent simply ignore tool problems in general: under automatic scoring it reports 58.1% of explicit failures, and the audit in Section 5.7 shows the true share is higher. Silent faults are flagged in 1.5% of faulted runs, and in no more than 6.7% for any of the twelve model and agent combinations.

5.2 How much of the pass-through could any agent avoid?

The 89% figure is partly fixed by the fault mix. A stale value or a 2% perturbation is plausible by design, so a single-source agent has nothing that separates it from the truth. A ×100 price is different, because a plausibility check can reject it. Of the 161 answer-changing faults faced by the plain agent, 33 (20.5%) include a unit-scale fault

Table 3: Unsafe answer rate (U) and fault detection rate (D) among faulted runs, in percent, by model, condition and agent, under automatic scoring. In the explicit condition the automatic U overstates the audited rate, and the per-model A-versus-B differences there (Qwen2.5-14B, Mistral-Small-24B) disappear after audit (Section 5.7).

	Silent				Explicit				Redundant source			
	Agent A		Agent B		Agent A		Agent B		Agent A		Agent B	
Model	U	D	U	D	U	D	U	D	U	D	U	D
Granite-4.1-8B	46.7	3.3	43.3	6.7	0.0	27.7	0.0	17.0	6.7	31.7	1.7	63.3
Qwen3-8B	38.3	0.0	38.3	0.0	2.1	94.7	4.3	86.2	38.3	0.0	8.3	25.0
Mistral-Nemo-12B	60.0	0.0	58.3	0.0	19.1	68.1	18.1	71.3	36.5	3.8	35.0	16.7
Qwen2.5-14B	36.7	3.3	36.7	1.7	12.8	52.1	2.0	69.7	27.1	13.6	6.8	37.3
Mistral-Small-24B	56.2	0.0	62.5	0.0	1.3	98.7	13.2	86.8	–	–	–	–
gpt-oss-20b	45.8	1.7	38.1	4.8	7.5	14.0	8.9	7.9	5.8	26.9	5.8	34.6

Table 4: Pass-through (%) on answer-changing faults, by whether a plausibility check could catch the fault in principle. Five models, runs paired by model, task and seed. n is the number of pairs.

	One source			Second source		
Fault class	n	A	B	n	A	B
Undetectable (stale, ±2%)	108	100.0	96.3	106	52.8	22.6
Detectable (×100 unit error)	30	53.3	43.3	28	21.4	10.7

and 128 do not. We call the first group detectable in principle and the second undetectable from one source. The split uses only the fault log, not agent behavior. A single-source agent that flags only values that look implausible therefore passes through at least 79.5% of answer-changing faults, whatever its model.

Table 4 splits the paired results this way. On undetectable faults the plain agent passes the corrupted value through in 126 of 126 pairs (six models) and the prompted agent in 96.8% (93.9 to 99.3). On detectable faults both agents still pass the corrupted value through about half of the time (48.5% and 45.5%), and the prompt effect is not significant (−3.0 points, CI −16.0 to +5.0, 3 versus 2 discordant pairs, $p = 1.0$). This class holds only 33 pairs, so a moderate effect cannot be excluded. The sharper claim is that the 89% figure is mostly a property of the faults, and that on the one class where an agent could have caught the error, neither agent does so reliably and the prompt does not change that.

A second source changes the picture for both classes. It makes the undetectable faults checkable, and pass-through on them falls from 100% to 52.8% for the plain agent and from 96.3% to 22.6% for the prompted agent (five models). The paired prompt effect on this class is −30.2 points (CI −38.6 to −21.7, 38 versus 6 discordant pairs, $p < 0.001$). On detectable faults the rates are 21.4% and 10.7% (28 pairs, $p = 0.51$).

5.3 A verification prompt alone helps little

Adding the verification prompt changes the pooled silent-condition unsafe rate from 46.5% to 44.8% and the detection rate from 1.5% to 2.4%. On paired runs the prompted agent's unsafe rate differs by −1.2 points (CI −3.0 to +0.6). The two agents reach the same outcome on 96.4% of the 331 paired faulted runs. The plain agent is unsafe where the prompted agent is not in 8 pairs, and the reverse occurs in 4 (McNemar $p = 0.39$). No single model shows a significant difference.

This pooled figure understates what the prompt could achieve, because about half of the faulted runs carry a fault that leaves the answer intact, and those runs are rarely unsafe under either agent. Restricted to the 161 pairs in which the fault changed the answer for both agents, the prompted agent is safer in 7 pairs and less safe in 2 (paired difference −3.1 points, CI −6.5 to 0.0, $p = 0.18$). On the five models with redundant-source data the point estimate is similar (pass-through 90.0% to 84.4%). We read this as a small effect at most rather than as no effect. The prompted agent still reports the corrupted value on 85.8% of answer-changing faults (76.1 to 94.3), and Section 5.2 shows why little more could be expected from a single source.

The prompt tells the agent to sanity-check values for plausibility and to cross-check with a second tool when one exists. With a single source, the first instruction is the only one the agent can act on, and a 2% perturbation or a value from the previous day is plausible by design. The prompt has a small cost. On clean tools it raises false alarms to 5.3% for Qwen2.5-14B and 1.5% for gpt-oss-20b, and leaves the other four at zero. Under explicit failures the two agents are indistinguishable (26 versus 26 discordant pairs, 6 versus 4 after audit).

5.4 A second source helps, and the prompt helps more with it

On the five models with redundant-source data, adding an uncorrupted second source lowers the unsafe rate of the plain agent from 45.5% to 23.0% (15.7 to 31.3) and of the prompted agent from 42.9% to 11.7% (7.3 to 16.7). Restricted to answer-changing faults, the pass-through rate falls from 90.0% to 46.3% for the plain agent and from 84.4% to 19.6% for the prompted agent. With the second source available, the prompted agent is now significantly safer than the plain agent. The paired difference is −12.5 points (CI −18.3 to −7.3), with 46 pairs favoring Agent B and 11 favoring Agent A ($p < 0.001$). Detection by hedge phrase rises from about 2% to 15.2% for Agent A and 35.4% for Agent B, over all faulted runs. This understates how often the answer discloses the problem (see below). False alarms stay rare with the second source: 0.1% (0.0 to 0.3) of the 988 unfaulted Agent A runs and 0.9% (0.2 to 1.7) of the 1,003

Table 5: Redundant-source condition: share of runs in which the agent called the secondary tool, and the unsafe rate (%) under silent faults without and with the second source.

	Calls secondary		Unsafe: silent → redundant	
Model	A	B	A	B
Granite-4.1-8B	77.9	92.4	46.7 → 6.7	43.3 → 1.7
Qwen3-8B	0.0	62.2	38.3 → 38.3	38.3 → 8.3
Mistral-Nemo-12B	71.9	100.0	60.0 → 36.5	58.3 → 35.0
Qwen2.5-14B	24.4	69.1	36.7 → 27.1	36.7 → 6.8
gpt-oss-20b	71.6	100.0	45.8 → 5.8	38.1 → 5.8

unfaulted Agent B runs. Comparing each agent's unsafe rate over its faulted runs, the B-minus-A difference is 8.7 points larger with the second source than without it (interaction CI −14.7 to −3.5). Section 5.8 shows that this interaction depends on one model.

Part of this interaction follows from the design, since the prompt explicitly asks the agent to call a second tool when one exists. What the result adds is that the prompt's other instructions (check plausibility, report uncertainty) achieve little when there is nothing to check against. Once a second source exists they do contribute. Granite-4.1-8B consults the second source with or without the prompt, and on answer-changing faults the prompt turns most of its silent repairs into flagged answers (7 repairs and 17 flags without it, 3 and 25 with it, Table 11).

Mechanism. Table 5 suggests why the prompt and the second source interact. Without the prompt, Granite-4.1-8B, Mistral-Nemo-12B and gpt-oss-20b call the secondary tool in 72 to 78% of runs, presumably because its description invites cross-checking. Qwen3-8B never calls it, and Qwen2.5-14B calls it in 24% of runs. For Qwen3-8B the plain agent's unsafe rate is unchanged by the second source (38.3% in both conditions), as expected if the tool is never used. The prompt raises secondary-tool use to 62% and 69% for these two models. They are the only two with a significant A-versus-B difference in this condition (20 versus 2 and 12 versus 0 discordant pairs, both significant after Bonferroni correction across five models). For the three models that already consult the second source, the prompt adds little to safety. For two of them (Granite-4.1-8B and gpt-oss-20b) the plain agent is already below 7% unsafe, which leaves little room. Pooled over agents, faulted runs that consulted the second source ended unsafe 11.9% of the time (8.4 to 15.7) and the others 26.4% of the time (15.2 to 40.9). This last comparison is observational, since the agent chooses when to consult.

Two stages of failure. Table 11 and Figure 2 classify every answer-changing faulted run in the redundant-source condition by what the agent did with the second source. An agent can fail to **acquire** the conflicting evidence by never querying it, or acquire it and fail to **reconcile** it by still reporting the corrupted value. This analysis uses audited labels. For the plain agent, 47 of 134 runs (35%) are acquisition failures and 11 (8%) are reconciliation failures. With the prompt, acquisition failures fall to 7 of 138 (5%), reconciliation failures rise slightly to 15 (11%), and the share of runs whose answer flags a problem rises from 31% to 54%. The two stages are model

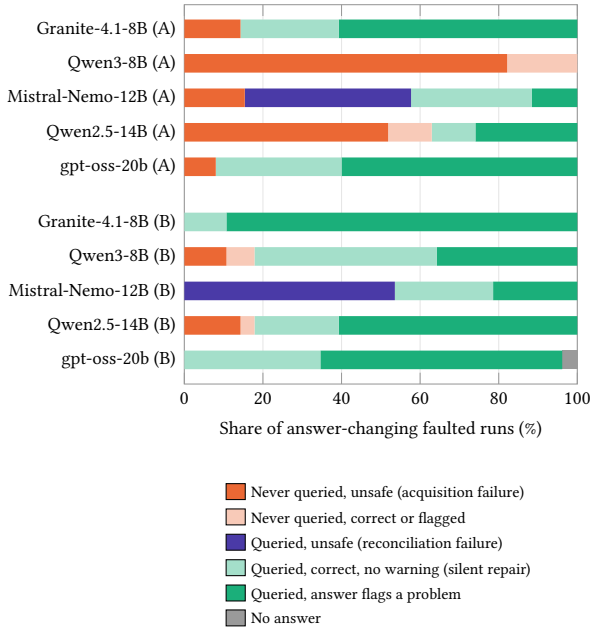


Figure 2: Failures with a second source arise at two different stages. Outcome of every answer-changing faulted run in the redundant-source condition, by model and agent (A on top, B below), using audited labels. Without the prompt, Qwen3-8B and Qwen2.5-14B fail mainly by never querying the second source. Mistral-Nemo-12B often queries it and still reports the corrupted value.

specific. Qwen3-8B and Qwen2.5-14B account for 37 of the 47 plain-agent acquisition failures, and all 26 reconciliation failures come from Mistral-Nemo-12B. This shows where each model's failures concentrate, which the unsafe rates alone do not, although each model and agent cell holds only 25 to 28 runs. A run counts as having queried the second source if it called the secondary tool at all, whatever the arguments.

Retrieved but not reconciled. Under automatic scoring, 34 answer-changing runs consulted the second source and still ended unsafe, 27 of them from Mistral-Nemo-12B. The audit confirms 26 of the 34, and all 26 are Mistral-Nemo-12B. Its prompted agent calls the second source in every run, yet it flags only 16.7% of faulted runs and its unsafe rate stays at 35.0%. In all 21 of its single-value failures, the answer states the corrupted primary value with no warning that the two sources disagreed. This model retrieves the conflicting evidence and then does not use it. We have not coded why, so we do not claim a mechanism such as anchoring on the first source. Redundancy also has a cost in calls. On the five models, the prompted agent averages 2.36 tool calls per run with the second source and 1.41 without it, against 1.84 and 1.30 for the plain agent.

Silent repair and a preference for the second tool. The second source is never wrong, so an agent that simply prefers the second tool would look like a careful verifier, and the traces cannot fully separate the two. Among the 64 answer-changing runs that queried

Table 6: Scripted check against the LLM agents in the redundant-source condition. A and B are the five models with audited labels. The scripted check runs all 131 tasks and both seeds. Flagged or disclosed counts an answer that states the disagreement without a hedge phrase.

	A	B	Script
Pass-through, answer-changing faults (%)	43.3	15.9	0.0
Flagged or disclosed, answer-changing faults (%)	44.0	74.6	100.0
Flagged, unfaulted runs (%)	0.1	0.9	0.0
Tool calls per run	1.84	2.36	2.61

the second source and ended correct with no hedge phrase, the secondary tool was the last call in 63 and the answer equals the last value read in 62. The 26 reconciliation failures are the exceptions. There the secondary was also the last call, and the agent still reported the earlier primary value. Many of the 64 do more than prefer the second tool. In 44 the answer states the disagreement or lists both values, which keyword scoring does not count as a flag. Counting those as disclosed, 44.0% (36.9 to 51.6) of Agent A's and 74.6% (66.9 to 81.8) of Agent B's answer-changing faulted runs end flagged or disclosed. The other 20 are silent by our rule (Appendix C), and on reading them, 18 mention no problem at all. These silent repairs are 9 of 134 Agent A runs and 11 of 138 Agent B runs. If the second source were corrupted as often as the primary, a preference for it would no longer reliably repair anything, so the reductions above are an upper bound.

5.5 Enforcing verification in the harness

The two failure stages suggest remedies that do not depend on model initiative. We test them without further model calls.

Scripted check. For each fact a task needs, a scripted policy calls the primary tool and its secondary twin, compares the two values within the scoring tolerance, and reports the value if they agree and "unable to confirm" if they do not. It runs through the same seeded injector and redundant-source policy as the LLM agents, over all 131 tasks and both seeds (262 runs). Its call order differs from that of the LLM agents, so its faults are matched to theirs by task and seed and not call by call. Where the primary-call sequence is the same they are identical. All 2,522 stored redundant-source runs with a matching sequence have the same logged corruptions as the scripted run, and 43 runs with a different sequence cannot be compared.

Of its 262 runs, 60 were faulted and 28 carried an answer-changing fault. It reported none of the 28 and flagged all of them. It also flagged 18 of the 32 faulted runs in which the fault did not change the answer, and none of the 202 unfaulted runs. It makes 2.61 tool calls per run, against 1.84 for the plain and 2.36 for the prompted LLM agent (Table 6). The zero is true by construction. The secondary is never corrupted, so agreement within tolerance implies that the primary is right and any disagreement is flagged. The result shows what enforcement costs, one extra call per fact and 18 flags on harmless faults, and that acquisition failures can be removed. It says nothing about how a harness copes with a second source that is itself wrong.

Table 7: Unsafe / detection rate (%) by realized fault type, five models pooled. n is the number of faulted Agent A runs in the silent condition, and "Chg." the share of them in which the fault changed the correct answer.

			Silent		Redundant source	
Fault	n	Chg.	A	B	A	B
Stale (1 period)	154	36	41 / 0	39 / 1	21 / 14	10 / 32
Perturbation (±2%)	65	69	71 / 0	68 / 0	34 / 30	20 / 56
Unit scale (×100)	30	83	43 / 13	37 / 10	19 / 4	14 / 24
Partial (holdings)	25	0	0 / 0	0 / 0	0 / 0	0 / 16
Two or more	25	60	56 / 4	52 / 10	33 / 17	12 / 36

Trace-level monitor. Acquisition is not the only stage. A reconciliation failure leaves the evidence in the trace, so a monitor can compare the final answer with the secondary tool's returned values without calling the model again. Applied to the stored runs, it flags all 26 reconciliation failures (all from Mistral-Nemo-12B). It flags none of the other 180 answer-changing runs that consulted the second source, using audited labels, and none of 1,403 further correct runs that consulted it. This follows from the clean secondary as well, but it shows that the check needs only the trace.

5.6 Fault types differ in visibility

Table 7 breaks results down by the fault that actually fired. A ±2% perturbation is invisible from a single source (71% unsafe, 0% detected) and becomes the most often detected fault once a second source exists (56% for Agent B). Stale values change the answer less often (36%). Of the 99 stale faults that leave the answer intact, 69 hit a comparison task, where the previous period's value did not flip the direction. The other 30 hit single-value tasks where the previous value differs from the current one by less than the scoring tolerance, such as a fed funds rate of 5.301 against 5.300. When a stale value does change the answer, it is passed through as reliably as a perturbation (55 of 55 runs). A ×100 unit error is the only single fault type that the plain agent ever flags from a single source (13%), consistent with the finding that implausible values are adopted less often [30]. It still leads to an unsafe answer in 43% of runs. In 10 of the 25 answer-changing unit-scale runs, the plain agent gives a correct answer without mentioning the anomaly. We call this **silent repair**. The answer is right, but a downstream reviewer has no record that the data was wrong. Partial-result faults never changed an answer, because in every realized case the dropped rows did not include the holding the question asked about. We keep these runs in all denominators as specified. Excluding them would raise the six-model pooled silent rate for Agent A from 46.5% to 51.2%.

5.7 How much does scoring error matter?

Keyword scoring can mistake a flagged answer for an unsafe one when the model words its flag in an unanticipated way. We read every answer the scorer labeled unsafe in the explicit (82) and redundant-source (99) conditions, and a seeded random sample of 60 of the 304 in the silent condition. Each was labeled as a confirmed unsafe answer, a missed flag, a scoring error on a correct answer, or a non-answer (an empty reply or leaked tool-call text).

Table 8: Pooled unsafe answer rate (%) under automatic scoring and after the manual audit. Explicit and redundant-source audits read every unsafe-labeled answer. The silent value is scaled by the confirmed fraction in a random sample of 60. Its range comes from the Wilson interval on that fraction and omits the sampling uncertainty of the rate itself (34.5 to 58.9 before the audit). Square brackets are task-clustered bootstrap intervals.

Quantity	Automatic	Audited
Silent, Agent A (6 models)	46.5	44.2 (40.2–45.7)
Explicit, Agent A (6 models)	7.3	2.9 [1.5, 4.6]
Explicit, Agent B (6 models)	7.5	2.5 [1.2, 3.8]
Redundant, Agent A (5 models)	23.0	21.2 [14.4, 28.7]
Redundant, Agent B (5 models)	11.7	10.0 [6.1, 14.5]
Redundant, A vs. B discordant	46 vs. 11	43 vs. 8

The audit reads only answers labeled unsafe, so it can lower unsafe rates but never raise them. A wrong answer that matched a hedge phrase without really flagging the problem would go unnoticed. This matters most for Agent B, whose prompt invites hedging. In the redundant-source condition, 10 Agent B answers and 2 Agent A answers are wrong but flagged. Even if every one of these flags were spurious, Agent B's unsafe rate would be 15.1% against 23.7% for Agent A. We also examined all 103 faulted Agent B runs in this condition whose answer carries a hedge flag (paper/audit/review_checks.json). The 10 that are wrong were read in full. All 10 say that the sources conflict or that the result is uncertain, and three of them reach a correct conclusion on rereading, so they are scoring errors and not unsafe answers. For the 93 correct ones, the trace shows a gap between the two sources beyond the scoring tolerance in 79. Of the other 14, 13 flag a real but small gap or a differing holdings list, and one reports a discrepancy between identical values. We found no spurious flag that hides an unsafe answer. These readings were made after the main audit by a single reader.

The audit separates the conditions sharply (Table 8). In the silent sample, 57 of 60 labels are confirmed. In the explicit condition only 30 of 82 are. Of the rest, 39 are abstentions the keyword list missed ("Sorry, I'm having trouble reaching the server right now"), 12 are empty replies from gpt-oss-20b, and one is a correct answer. The automatic explicit rates are therefore inflated by a factor of 2.5 (Agent A) to 3 (Agent B), and the audited gap between silent and explicit faults is larger, not smaller. The redundant-source labels are 90% confirmed, and the A-versus-B difference remains highly significant ($p < 0.001$). The errors are uneven across models, which is what makes them dangerous. The apparent per-model A-versus-B differences under explicit failures in Table 3 come entirely from phrasings the scorer missed for one agent and not the other: 11 abstentions by Qwen2.5-14B Agent A and 10 by Mistral-Small-24B Agent B. Before freezing the scorer we had already fixed three problems of this kind: contracted phrasings ("couldn't retrieve"), which had misscored 87% of one model's faulted explicit-failure runs as unsafe, typographic apostrophes and hyphens, and comparison answers phrased as "did increase".

The confirmed explicit-condition errors are informative in their own right. Mistral-Nemo-12B often answers a comparison question with the one value it did retrieve, and gpt-oss-20b sometimes replaces the failed lookup with a figure from its own memory, such as a Federal Reserve target range.

5.8 Robustness to subsetting

A reviewer might ask whether the headline contrasts are driven by one model, seed, task family or fault type. Table 10 (Appendix B) recomputes them leaving out each model in turn, within each seed, within each task family and answer type, and without runs that contain a unit-scale fault. The first three results are stable. The gap between silent and explicit faults is positive in every subset and the pass-through rate of answer-changing faults is at least 77% in every subset. Without a second source, the paired effect of the prompt lies between −2.0 and 0.0 points in every subset. With a second source, it is between −5.0 and −18.4 points, and every leave-one-model-out interval is below zero. Three qualifications apply. The gap between silent and explicit faults is concentrated in market-data tasks (58.6 points) and single-value tasks (54.6 points), and its interval includes zero for portfolio tasks (12.0, CI −5.3 to 35.3). This is expected because the unit-scale and perturbation faults fire only on market-data calls. The prompt effect with a second source reaches zero on portfolio tasks. Most important, the interaction between prompt and second source depends on Qwen3-8B. It stays significantly negative when any other single model is dropped (−5.7 to −12.6 points), but without Qwen3-8B it is −3.0 (CI −8.6 to 1.8), and it is not significant within comparison or portfolio tasks. We therefore read the interaction as suggestive. The more robust statement is that a second source reduces unsafe answers and that the prompt then adds a further reduction.

6 Discussion

A hierarchy of controls. Our results correspond to four levels of control over a tool-using agent. **Control 0**, trusting the tool output, fails (Section 5.1). **Control 1**, instructing the model to be skeptical, helps little, and cannot help much when four in five harmful faults look plausible from one source (Section 5.2). **Control 2**, providing independent evidence, roughly halves the unsafe rate of the plain agent and cuts it by three quarters for the prompted agent (Section 5.4). It still depends on the model using the evidence, which two models often do not and one does badly. **Control 3**, having the harness fetch the second source and check the answer against it, removes that dependence. In our best-case design it left no unsafe answer (Section 5.5). The instruction is not useless, since it is what led Qwen3-8B to consult the second source at all, but as a control it was a weak substitute for independent evidence. For practitioners this suggests spending reliability effort on independent evidence (a second vendor, a reference value, a reconciliation tool) and on enforcing its use, rather than relying on cautionary prompt text alone. We tested one prompt wording on models of at most 24B parameters, so this is a statement about this kind of prompt, not about instructions in general.

Evidence alone is not sufficient, and the two failure stages in Figure 2 point to what a system should add. Some models do not use evidence they are offered, so the harness should query the second

Table 9: Findings and their implications for deploying financial agents.

Finding	Implication
Answer-changing silent faults are reported without warning in 89% of runs	A successful call that passes schema validation is not evidence that the value is right
Four in five answer-changing faults cannot be seen from one source, and a verification prompt alone gave at most a small gain	A cautionary prompt alone should not be counted on as a reliability control
An independent source cuts unsafe answers substantially	High-impact facts should have an independent source to check against
Some models never query the second source	Verification should not depend entirely on model initiative, so the harness should query it
One model retrieves the conflict and still reports the corrupted value	Check the final answer against the evidence in the trace
A scripted two-source check left no unsafe answer but flagged 18 harmless faults	Enforcement is cheap, but it needs a second source that is not itself wrong
Keyword scoring mislabeled 52 of 82 explicit-failure answers it called unsafe	Monitoring and evaluation of agents needs semantic validation of answers

source itself. One model retrieves both sources and still reports the corrupted one, so the final answer should be checked against the evidence in the trace. Table 9 lists the implications we draw from each result.

Audit trails and model risk. Silent repair is an audit-trail failure. In 18 of the 64 correct answers that consulted the second source and carried no hedge phrase, nothing in the answer says the primary source was wrong, so a reader cannot tell that a fault occurred. Model risk guidance stresses input data quality and documentation [2], and a deployment held to it would need the discrepancy recorded even when the final figure is right. The trace-level monitor is the matching evidence requirement, since it ties each reported figure to the source values that support it.

Relation to prior findings. ToolMaze reports partial gains (1.5 to 20.8 points) from a failure-aware prompt [40], while we find at most a small gain without a second source. The settings differ in models (larger open and proprietary models versus at most 24B), tasks (multi-path tool workflows versus lookups) and fault generation (LLM-written versus rule-based). One reconciliation consistent with our results is that multi-path tasks can supply alternative evidence for the prompt to act on, while our single-source condition supplies none. We have not tested this directly.

What the results do not support. We make no claim about which model family is more robust. The study is not powered for that, fault draws are shared across models, and Mistral-Small-24B's serving defect changes which tasks it completes. We also make no claim about frontier models, long multi-step workflows, or redundancy that is itself imperfect. Our second source is never corrupted, which is the most favorable case. The scripted check and the trace monitor owe their perfect scores to that choice and say nothing about imperfect redundancy. The mechanism account rests on five models, two of which drive the interaction, and the interaction is not significant once Qwen3-8B is dropped (Section 5.8). It should be read as a hypothesis supported by this data rather than a general law.

Evaluation practice. Two conclusions we drew during development reversed once scoring was audited: an apparent advantage of the verification prompt under explicit failures, and an apparent reversal of that advantage for another model. Each came from the scorer missing one model's phrasing for one agent. Benchmarks that score abstention or hedging with keyword rules should report a manual audit stratified by condition and model, because the rules fail unevenly across both.

7 Limitations

The tasks are simple lookups and two-date comparisons over synthetic snapshots of three tickers, three macro series and three portfolios. Real analyst workflows are longer and the data less regular, which may make faults harder or easier to notice. All models are open-weight and at most 24B parameters, decoded greedily with two fault-draw seeds per condition, and four of the six are quantized, which may affect instruction following. Fault probabilities are illustrative rather than calibrated to a real data feed. Explicit failures fire on more runs than silent faults (36% versus 22%), so the two conditions differ in sample size. Detection is measured from the final answer only, and the answer-changing replay assumes the agent made the same calls as the clean trace (true for all but 3 of 1,240 faulted runs). The manual audit was performed by the author as a single annotator, without double annotation and without blinding to agent or condition. It read only answers labeled unsafe, and in the silent condition it read a simple random sample rather than every answer. We tested a single wording of the verification prompt. Our second source is never wrong, so a policy that always prefers it scores as a verifier, and the redundancy gains are an upper bound. The split of correct, unflagged answers into disclosed and silent uses a keyword rule that only the author's reading has checked. The detectable-fault class holds 33 pairs, which limits what it can show. Mistral-Small-24B lacks redundant-source data and has a 26% exclusion rate, concentrated in two-call tasks and dependent on whether the agent attempted a second call.

8 Future Work

Each limitation above suggests a next experiment. The most direct is to make the second source imperfect. Our redundant source is never corrupted, which is the best case. Real vendors share upstream feeds, so a second source can be stale in the same way as the first, or can disagree by a small and legitimate amount. Varying the correlation and the error rate of the second source would show how much of the gain survives. A second extension is a suite of multi-step financial workflows, in which a fault early in the chain feeds later calculations and the replay used here no longer applies. Third, we would deepen the retrieve-versus-reconcile analysis. The

two-stage split locates where each model fails but not why. Coding the Mistral-Nemo-12B traces, varying the order and the labeling of the two sources, and testing whether stronger models close the gap would separate source-order bias from a general failure to compare. Larger and proprietary models belong in the same study, since we cannot say whether the null result for the verification prompt holds beyond 24B parameters.

The last extension is to enforce verification in the harness with a model in the loop. The scripted check in Section 5.5 flags a disagreement but cannot say which source is right, and it was never tested against a wrong second source. A structured reliability agent (Agent C in the FinAgentFault design) would query the second source itself, compare the values and report claims with provenance before answering. This also gives a concrete basis for governance requirements, such as which facts must be independently verified and what a reconciliation record must contain.

9 Conclusion

FinAgentFault isolates a failure mode that standard agent evaluations leave out: tool output that is well-formed, plausible and wrong. Across six open-weight models, a silent fault that changes the answer is passed through to the user about nine times in ten, almost never with a warning. Most such faults cannot be seen from a single source, and telling the agent to be skeptical changes little. Giving it a second source helps far more, and the instruction to verify adds a further reduction once that source exists. The failures that remain are of two kinds, not acquiring the second source and not reconciling it, and a scripted check removes both when the second source is clean. We outline next steps, led by imperfect redundant sources, in Section 8.

References

[1] Antoine Bigeard, Langston Nashold, Rayan Krishnan, and Shirley Wu. 2025. Finance Agent Benchmark: Benchmarking LLMs on Real-world Financial Research Tasks. *arXiv preprint arXiv:2508.00828* (2025).

[2] Board of Governors of the Federal Reserve System and Office of the Comptroller of the Currency. 2011. Supervisory Guidance on Model Risk Management. SR Letter 11-7.

[3] Jiawei Chen, Hongyu Lin, Xianpei Han, and Le Sun. 2024. Benchmarking Large Language Models in Retrieval-Augmented Generation. In *Proceedings of the AAAI Conference on Artificial Intelligence.*

[4] Zhiyu Chen, Wenhu Chen, Charese Smiley, Sameena Shah, Iana Borova, Dylan Langdon, Reema Moussa, Matt Beane, Ting-Hao Huang, Bryan Routledge, and William Yang Wang. 2021. FinQA: A Dataset of Numerical Reasoning over Financial Data. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing (EMNLP).*

[5] Edoardo Debenedetti, Jie Zhang, Mislav Balunović, Luca Beurer-Kellner, Marc Fischer, and Florian Tramèr. 2024. AgentDojo: A Dynamic Environment to Evaluate Prompt Injection Attacks and Defenses for LLM Agents. In *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track.*

[6] Shehzaad Dhuliawala, Mojtaba Komeili, Jing Xu, Roberta Raileanu, Xian Li, Asli Celikyilmaz, and Jason Weston. 2024. Chain-of-Verification Reduces Hallucination in Large Language Models. In *Findings of the Association for Computational Linguistics: ACL 2024.*

[7] Harish Dattatraya Dixit, Sneha Pendharkar, Matt Beadon, Chris Mason, Tejasvi Chakravarthy, Bharath Muthiah, and Sriram Sankar. 2021. Silent Data Corruptions at Scale. *arXiv preprint arXiv:2102.11245* (2021).

[8] Bradley Efron and Robert J. Tibshirani. 1993. *An Introduction to the Bootstrap.* Chapman and Hall/CRC.

[9] Zhibin Gou, Zhihong Shao, Yeyun Gong, Yelong Shen, Yujiu Yang, Nan Duan, and Weizhu Chen. 2024. CRITIC: Large Language Models Can Self-Correct with Tool-Interactive Critiquing. In *International Conference on Learning Representations (ICLR).*

[10] Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. 2023. Not What You've Signed Up For: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection. In *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security (AISec).*

[11] Bhaskar Gurram. 2026. Auditing Automated Evaluation, Error Propagation, and Runtime Mitigation in Tool-Using Language Agents. *arXiv preprint arXiv:2604.16706* (2026).

[12] Jie Huang, Xinyun Chen, Swaroop Mishra, Huaixiu Steven Zheng, Adams Wei Yu, Xinying Song, and Denny Zhou. 2024. Large Language Models Cannot Self-Correct Reasoning Yet. In *International Conference on Learning Representations (ICLR).*

[13] IBM. 2026. Granite 4.1 Language Models. https://huggingface.co/ibm-granite/granite-4.1-8b.

[14] Pranab Islam, Anand Kannappan, Douwe Kiela, Rebecca Qian, Nino Scherrer, and Bertie Vidgen. 2023. FinanceBench: A New Benchmark for Financial Question Answering. *arXiv preprint arXiv:2311.11944* (2023).

[15] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient Memory Management for Large Language Model Serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP).*

[16] Ji Lin, Jiaming Tang, Haotian Tang, Shang Yang, Wei-Ming Chen, Wei-Chen Wang, Guangxuan Xiao, Xingyu Dang, Chuang Gan, and Song Han. 2024. AWQ: Activation-aware Weight Quantization for On-Device LLM Compression and Acceleration. In *Proceedings of Machine Learning and Systems (MLSys).*

[17] Xiao Liu, Hao Yu, Hanchen Zhang, Yifan Xu, Xuanyu Lei, Hanyu Lai, Yu Gu, Hangliang Ding, Kaiwen Men, Kejuan Yang, Shudan Zhang, Xiang Deng, Aohan Zeng, Zhengxiao Du, Chenhui Zhang, Sheng Shen, Tianjun Zhang, Yu Su, Huan Sun, Minlie Huang, Yuxiao Dong, and Jie Tang. 2024. AgentBench: Evaluating LLMs as Agents. In *International Conference on Learning Representations (ICLR).*

[18] Shayne Longpre, Kartik Perisetla, Anthony Chen, Nikhil Ramesh, Chris DuBois, and Sameer Singh. 2021. Entity-Based Knowledge Conflicts in Question Answering. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing (EMNLP).*

[19] Quinn McNemar. 1947. Note on the Sampling Error of the Difference Between Correlated Proportions or Percentages. *Psychometrika* 12, 2 (1947), 153–157.

[20] Mistral AI. 2024. Mistral NeMo. https://mistral.ai/news/mistral-nemo.

[21] Mistral AI. 2025. Mistral Small 3. https://mistral.ai/news/mistral-small-3.

[22] OpenAI. 2025. gpt-oss-120b & gpt-oss-20b Model Card. *arXiv preprint arXiv:2508.10925* (2025).

[23] Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, Sihan Zhao, Lauren Hong, Runchu Tian, Ruobing Xie, Jie Zhou, Mark Gerstein, Dahai Li, Zhiyuan Liu, and Maosong Sun. 2024. ToolLLM: Facilitating Large Language Models to Master 16000+ Real-world APIs. In *International Conference on Learning Representations (ICLR).*

[24] Qwen Team. 2024. Qwen2.5 Technical Report. *arXiv preprint arXiv:2412.15115* (2024).

[25] Qwen Team. 2025. Qwen3 Technical Report. *arXiv preprint arXiv:2505.09388* (2025).

[26] Yangjun Ruan, Honghua Dong, Andrew Wang, Silviu Pitis, Yongchao Zhou, Jimmy Ba, Yann Dubois, Chris J. Maddison, and Tatsunori Hashimoto. 2024. Identifying the Risks of LM Agents with an LM-Emulated Sandbox. In *International Conference on Learning Representations (ICLR).*

[27] Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. 2023. Toolformer: Language Models Can Teach Themselves to Use Tools. In *Advances in Neural Information Processing Systems (NeurIPS).*

[28] Ruipeng Wang, Yuxin Chen, Yukai Wang, Chang Wu, Junfeng Fang, Xiaodong Cai, Qi Gu, Hui Su, An Zhang, Xiang Wang, Xunliang Cai, and Tat-Seng Chua. 2026. AgentNoiseBench: Benchmarking Robustness of Tool-Using LLM Agents Under Noisy Condition. *arXiv preprint arXiv:2602.11348* (2026).

[29] Bingbing Wen, Jihan Yao, Shangbin Feng, Chenjun Xu, Yulia Tsvetkov, Bill Howe, and Lucy Lu Wang. 2025. Know Your Limits: A Survey of Abstention in Large Language Models. *Transactions of the Association for Computational Linguistics* 13 (2025), 529–556.

[30] Kevin Wu, Eric Wu, and James Zou. 2024. ClashEval: Quantifying the Tug-of-War between an LLM's Internal Prior and External Evidence. In *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track.*

[31] Shijie Wu, Ozan Irsoy, Steven Lu, Vadim Dabravolski, Mark Dredze, Sebastian Gehrmann, Prabhanjan Kambadur, David Rosenberg, and Gideon Mann. 2023. BloombergGPT: A Large Language Model for Finance. *arXiv preprint arXiv:2303.17564* (2023).

[32] Guangxuan Xiao, Ji Lin, Mickael Seznec, Hao Wu, Julien Demouth, and Song Han. 2023. SmoothQuant: Accurate and Efficient Post-Training Quantization for Large Language Models. In *International Conference on Machine Learning (ICML).*

[33] Jian Xie, Kai Zhang, Jiangjie Chen, Renze Lou, and Yu Su. 2024. Adaptive Chameleon or Stubborn Sloth: Revealing the Behavior of Large Language Models

Table 10: Headline contrasts recomputed on subsets, in percentage points with task-clustered 95% intervals. Silent vs. explicit is the silent minus explicit unsafe rate for Agent A. The two prompt columns are the paired B minus A difference in unsafe rate without and with a second source. Interaction is the B minus A difference in unsafe rate with a second source minus the same difference without it, on the five models with second-source data. It is computed from each agent's unsafe rate over its faulted runs, not from the paired columns. Pass-through is the unsafe rate of Agent A on answer-changing silent faults. Subsets that remove a model also remove it from every contrast.

Subset	Silent vs. explicit	Prompt, no 2nd source	Prompt, 2nd source	Interaction	Pass-through
All data (main analysis)	39.2 [26.9, 51.9]	−1.2 [−3.0, 0.6]	−12.5 [−18.3, −7.3]	−8.7 [−14.7, −3.5]	89.4 [80.3, 97.9]
Without Granite-4.1-8B	37.6 [25.3, 50.5]	−0.7 [−2.9, 1.4]	−14.6 [−22.2, −7.9]	−10.7 [−18.2, −4.2]	87.2 [76.1, 97.4]
Without Qwen3-8B	39.9 [27.8, 52.7]	−1.5 [−3.7, 0.8]	−7.8 [−12.4, −3.4]	−3.0 [−8.6, 1.8]	91.0 [82.9, 98.3]
Without Mistral-Nemo-12B	38.7 [26.0, 51.5]	−1.1 [−3.3, 1.1]	−14.5 [−21.0, −8.6]	−11.5 [−18.0, −5.6]	88.7 [78.5, 98.1]
Without Qwen2.5-14B	42.5 [30.3, 55.2]	−1.5 [−3.7, 0.8]	−10.4 [−15.5, −5.7]	−5.7 [−11.5, −0.6]	91.7 [84.3, 98.3]
Without Mistral-Small-24B	37.2 [25.6, 49.6]	−2.0 [−4.1, 0.0]	−12.5 [−18.3, −7.3]	−8.7 [−14.7, −3.5]	90.0 [81.4, 97.6]
Without gpt-oss-20b	39.4 [27.5, 51.9]	−0.4 [−2.2, 1.5]	−15.2 [−22.4, −8.6]	−12.6 [−20.3, −6.1]	88.0 [77.2, 97.8]
Seed 0 only	46.4 [27.9, 63.6]	−0.7 [−3.4, 2.6]	−13.1 [−20.8, −6.0]	−10.8 [−18.9, −3.2]	92.8 [81.8, 100.0]
Seed 1 only	33.2 [19.1, 49.0]	−1.7 [−4.3, 0.6]	−12.1 [−19.1, −5.5]	−7.0 [−14.7, −0.4]	85.9 [72.6, 97.5]
Market-data tasks only	58.6 [43.3, 74.1]	−1.6 [−5.0, 1.5]	−14.9 [−22.3, −8.1]	−10.3 [−19.2, −2.3]	86.5 [74.8, 96.6]
Macro tasks only	23.5 [0.6, 49.0]	0.0 [0.0, 0.0]	−9.2 [−17.7, −1.6]	−9.5 [−18.7, −1.9]	100.0 [100.0, 100.0]
Portfolio tasks only	12.0 [−5.3, 35.3]	−1.4 [−4.5, 0.0]	−10.0 [−24.2, 0.0]	−3.3 [−18.1, 8.7]	100.0 [100.0, 100.0]
Single-value tasks only	54.6 [38.2, 72.2]	−1.6 [−4.2, 0.5]	−18.4 [−26.4, −10.7]	−14.0 [−23.4, −6.5]	92.9 [83.3, 100.0]
Comparison tasks only	18.1 [3.8, 34.5]	−0.7 [−3.9, 2.4]	−5.0 [−10.3, −0.9]	−1.7 [−9.1, 5.0]	77.1 [48.0, 100.0]
Without unit-scale runs	40.6 [27.1, 54.4]	−1.4 [−3.1, 0.0]	−13.6 [−19.8, −8.0]	−10.1 [−16.6, −4.5]	100.0 [100.0, 100.0]

in Knowledge Conflicts. In *International Conference on Learning Representations (ICLR)*.

[34] Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik Narasimhan. 2024. τ-bench: A Benchmark for Tool-Agent-User Interaction in Real-World Domains. *arXiv preprint arXiv:2406.12045* (2024).

[35] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2023. ReAct: Synergizing Reasoning and Acting in Language Models. In *International Conference on Learning Representations (ICLR)*.

[36] Junjie Ye, Sixian Li, Guanyu Li, Caishuang Huang, Songyang Gao, Yilong Wu, Qi Zhang, Tao Gui, and Xuanjing Huang. 2024. ToolSword: Unveiling Safety Issues of Large Language Models in Tool Learning Across Three Stages. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*.

[37] Junjie Ye, Yilong Wu, Songyang Gao, Caishuang Huang, Sixian Li, Guanyu Li, Xiaoran Fan, Qi Zhang, Tao Gui, and Xuanjing Huang. 2024. RoTBench: A Multi-Level Benchmark for Evaluating the Robustness of Large Language Models in Tool Learning. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing (EMNLP)*.

[38] Qiusi Zhan, Zhixiang Liang, Zifan Ying, and Daniel Kang. 2024. InjecAgent: Benchmarking Indirect Prompt Injections in Tool-Integrated Large Language Model Agents. In *Findings of the Association for Computational Linguistics: ACL 2024*.

[39] Yuxiang Zhang, Jing Chen, Junjie Wang, Yaxin Liu, Cheng Yang, Chufan Shi, Xinyu Zhu, Zihao Lin, Hanwen Wan, Yujiu Yang, Tetsuya Sakai, Tian Feng, and Hayato Yamana. 2024. ToolBeHonest: A Multi-level Hallucination Diagnostic Benchmark for Tool-Augmented Large Language Models. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. 11388–11422.

[40] Dongsheng Zhu, Xuchen Ma, Yucheng Shen, Xiang Li, Yukun Zhao, Shuaiqiang Wang, Lingyong Yan, and Dawei Yin. 2026. When Tools Fail: Benchmarking Dynamic Replanning and Anomaly Recovery in LLM Agents. *arXiv preprint arXiv:2606.05806* (2026).

[41] Fengbin Zhu, Wenqiang Lei, Youcheng Huang, Chao Wang, Shuo Zhang, Jiancheng Lv, Fuli Feng, and Tat-Seng Chua. 2021. TAT-QA: A Question Answering Benchmark on a Hybrid of Tabular and Textual Content in Finance. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics (ACL)*.

[42] Junru Zhu, Shiming Xie, Aime Lu Fan Chen, Xiaoqing Ding, Chunxin Tang, Ruoyu Qi, and Yulang Fei. 2026. Failure-Transparent Agents: Benchmarking Post-Failure Reporting in Tool-Using Language Models. *arXiv preprint arXiv:2609.35732* (2026).

A Agent B system prompt

> You are a careful financial research assistant. A tool call can succeed and return well-formed data that is still wrong: it may be out of date, mis-scaled (for example cents reported as dollars), truncated, about the wrong entity, or in conflict with another source. Never assume tool output is correct just because the call did not error.
>
> Before you commit to a final answer:
>
> - Sanity-check every high-impact number against what is plausible for that quantity and against any other evidence you have gathered.
> - When more than one tool can supply the same fact, call a second tool and compare the results. If independent sources disagree, treat the fact as unconfirmed.
> - Confirm the data actually answers the question asked: the right entity, the right date, and units that make sense.
>
> If you cannot confirm a fact, or the evidence conflicts or looks out of date, say so plainly and explain what is uncertain instead of presenting a confident answer. Reporting a problem is better than reporting a plausible-looking wrong value.

B Supporting tables

Table 10 supports Section 5.8 and Table 11 supports the two-stage analysis in Section 5.4. Both are generated by experiments/analyze_paper.py.

C Definitions for the additional analyses

Detectable in principle. A faulted run is detectable if its logged fault types include a unit-scale fault. All other answer-changing faults (stale values, ±2% perturbations and their combinations) are undetectable from one source.

Table 11: Outcome counts for answer-changing faulted runs in the redundant-source condition (audited labels). The first two outcome columns are runs that never queried the second source, the next three are runs that did. "Repair" means correct with no hedge phrase, and 44 of its 64 answers still state the disagreement or both values (Section 5.4).

			Not queried		Queried			
Model	Ag.	n	Unsafe	Safe	Unsafe	Repair	Flag	None
Granite-4.1-8B	A	28	4	0	0	7	17	0
Granite-4.1-8B	B	28	0	0	0	3	25	0
Qwen3-8B	A	28	23	5	0	0	0	0
Qwen3-8B	B	28	3	2	0	13	10	0
Mistral-Nemo-12B	A	26	4	0	11	8	3	0
Mistral-Nemo-12B	B	28	0	0	15	7	6	0
Qwen2.5-14B	A	27	14	3	0	3	7	0
Qwen2.5-14B	B	28	4	1	0	6	17	0
gpt-oss-20b	A	25	2	0	0	8	15	0
gpt-oss-20b	B	26	0	0	0	9	16	1

Disclosed. An answer that the audited scoring treats as correct and unflagged is disclosed if it states the corrupted primary value as well as the right one, or contains any of these stems: differ, disagree, conflict, mismatch, discrepan, inconsisten, do not agree or match, unconfirm, implausib, scaling, unusual, incorrect, erroneous, misplaced, formatting, plausible, alternate, or a ×100 factor. Otherwise it is silent. We hand-read the 20 silent answers in the redundant-source condition (paper/audit/review_checks.json).

Scripted check. For each request in the task's clean tool trace, call the primary tool and its secondary twin and extract the asked-about field. If the two values agree within relative 10^{-3} or absolute 10^{-2} for every request, report the answer implied by the primary values. Otherwise report "Unable to confirm: the primary and secondary sources disagree."

Trace monitor. For each run that called the secondary tool for every request in the clean trace, derive the answer implied by the last secondary value of each request and flag the run if the final answer does not state it and carries no flag. Runs that did not call the secondary tool for every request are not evaluated.

D Reproducibility

The code, configs, stored runs and audit labels will be released upon acceptance. The paths below refer to that release. Hugging Face checkpoints:

- ibm-granite/granite-4.1-8b
- Qwen/Qwen3-8B
- casperhansen/mistral-nemo-instruct-2407-awq
- Qwen/Qwen2.5-14B-Instruct-AWQ
- RedHatAI/Mistral-Small-24B-Instruct-2501-quantized.w8a8
- openai/gpt-oss-20b

Fault policies are in configs/faults/: mvp.yaml for silent faults, baseline1.yaml for explicit failures and condition6_redundant.yaml for the redundant source. The script experiments/analyze_paper.py regenerates the main tables, the figure data and paper/generated/numbers.json from the stored runs, including the answer-changing replay. The script experiments/analyze_review.py regenerates the scripted check, the trace monitor, the fault-class split and the headline table and figure data (paper/generated/review_numbers.json). The files paper/audit/hedge_audit.json and paper/audit/review_checks.json contain every manual audit label.